

Estructura de Datos II

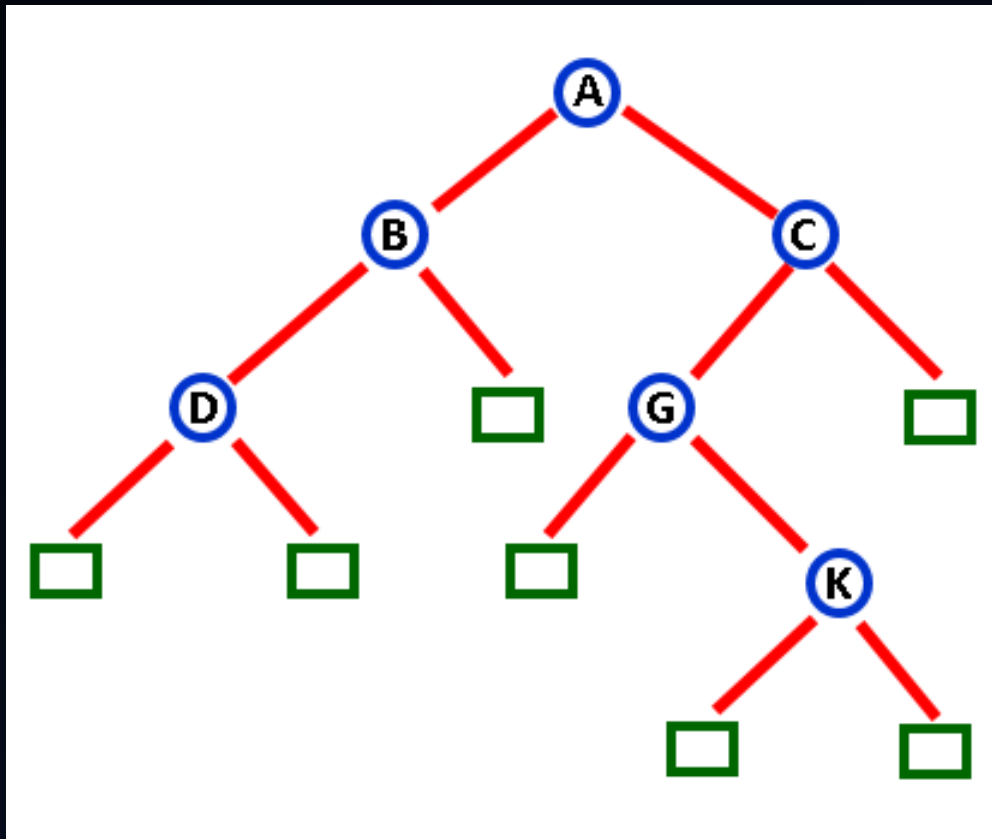
Arboles

10. Longitud de Camino Ponderado o con Pesos

Árboles Binarios Extendidos - Longitud de Camino

Ejemplo:

Dado el siguiente árbol binario extendido, encontrar la longitud de camino interna (LCI) y la longitud de camino externa (LCE).



$$LCI = 0 + 1 + 2 + 1 + 2 + 3 = 9$$

$$LCE = 3 + 3 + 2 + 3 + 4 + 4 + 2 = 21$$

10. Longitud de Camino Ponderado o con Pesos

Supongamos que T es un Árbol Binario Extendido, con NE nodos externos y supongamos que cada nodo externo tiene asignado un peso (positivo).

La longitud de camino ponderado o con peso (LP) del árbol binario extendido T , se define como la sumatoria de las longitudes de camino de los nodos externos multiplicada por sus pesos, mediante la siguiente formula:

$$LP = P_1 L_1 + P_2 L_2 + P_3 L_3 + \dots + P_n L_n$$

Donde:

$P_i \rightarrow$ corresponde al peso de los nodos externos NE .

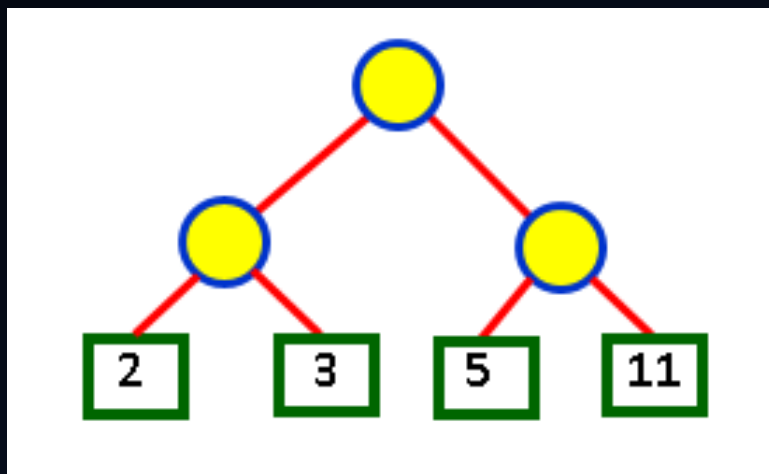
$L_i \rightarrow$ corresponde a la longitud de camino del nodo externo NE .

10. Longitud de Camino Ponderado o con Pesos

Ejemplo:

Encontrar la longitud de camino ponderado o con peso, al siguiente árbol binario extendido T_1

T_1



$$L P_{T_1} = P_1 L_1 + P_2 L_2 + P_3 L_3 + P_4 L_4$$

$$L P_{T_1} = (2) * (2) + (3) * (2) + (5) * (2) + (11) * (2)$$

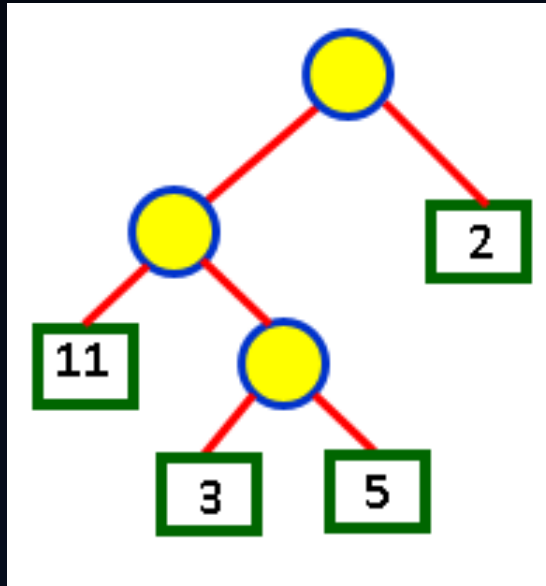
$$L P_{T_1} = 42$$

10. Longitud de Camino Ponderado o con Pesos

Ejemplo:

Encontrar la longitud de camino ponderado o con peso, al siguiente árbol binario extendido T_2

T_2



$$L P_{T_2} = P_1 L_1 + P_2 L_2 + P_3 L_3 + P_4 L_4$$

$$L P_{T_2} = (11) * (2) + (3) * (3) + (5) * (3) + (2) * (1)$$

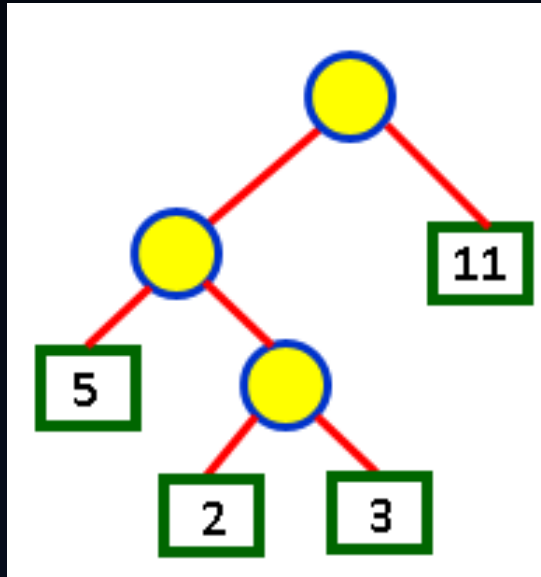
$$L P_{T_2} = 48$$

10. Longitud de Camino Ponderado o con Pesos

Ejemplo:

Encontrar la longitud de camino ponderado o con peso, al siguiente árbol binario extendido T_3

T_3



$$L P_{T_3} = P_1 L_1 + P_2 L_2 + P_3 L_3 + P_4 L_4$$

$$L P_{T_3} = (5) * (2) + (2) * (3) + (3) * (3) + (11) * (1)$$

$$L P_{T_3} = 36$$

11. Algoritmo de Huffman

Es un algoritmo que se usa para la compresión o encriptación de datos, desarrollado por David Huffman en 1952.

Recordemos que cada carácter se identifica por un código binario de 8 bits.

Este algoritmo se basa en asignar códigos de distintas longitudes, menores a 8 de bits, a cada uno de los caracteres de un archivo.

Si se asignan códigos más cortos a los caracteres que aparecen más a menudo se consigue una compresión del archivo.

El algoritmo consiste en la creación de un árbol binario extendido, donde cada uno de los caracteres del archivo, estará en un nodo externo.

Este árbol binario extendido es construido de tal forma que siguiéndolo desde la raíz a cada uno de sus nodos externos o nodos terminales, se obtiene el código de Huffman asociado.

11. Algoritmo de Huffman

9.1 Árbol Binario Extendido con Longitud de Camino con Peso Mínimo

Este algoritmo permite encontrar, de entre todos los árboles binarios extendidos con n nodos externos y con n pesos dados, un árbol binario extendido con longitud de camino con peso mínimo.

Pasos para construir el Árbol Binario Extendido con Peso Mínimo

Suponga que nos dan una lista de n pesos:

$$P_1, P_2, P_3, \dots, P_n$$

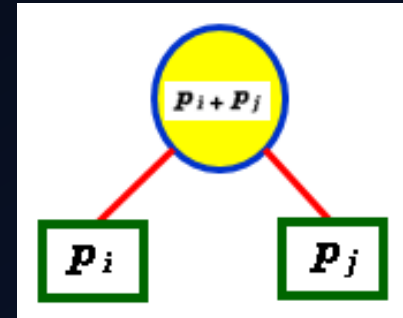
- Lo primero que tenemos que hacer es ordenar la lista de n pesos, de menor a mayor.
- De entre todos los n pesos dados, se encuentran los dos pesos mínimos; p_i y p_j
- Sustituimos los dos pesos mínimos encontrados p_i y p_j por un nuevo nodo externo $p_i + p_j$, luego de esta operación nos quedaran $n - 1$ pesos.

11. Algoritmo de Huffman

9.1 Árbol Binario Extendido con Longitud de Camino Ponderado Mínimo

Pasos para construir el Árbol Binario Extendido con Peso Mínimo

- d) Luego comenzamos a construir nuestro árbol binario extendido T^1 con longitud de camino con peso mínimo, para los $n - 1$ pesos.
- e) En el nuevo árbol binario extendido T^1 sustituimos el nuevo nodo externo $p_i + p_j$ por el subárbol:.



- f) Se continua con el mismo procedimiento hasta que terminemos de unir todos los nodos externos en un nuevo árbol binario extendido con longitud de camino con peso mínimo.

11. Algoritmo de Huffman

11.1 Árbol Binario Extendido con Longitud de Camino con Peso Mínimo

Ejemplo:

Suponga que A, B, C, D, y E son cinco datos y suponga que tienen asignado los siguientes pesos:

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
8	4	10	7	5

1. Ordenamos la lista de menor a mayor.

4	5	7	8	10
---	---	---	---	----

2. Seleccionamos los dos pesos mas pequeños o mínimos.

4	5	7	8	10
---	---	---	---	----

11. Algoritmo de Huffman

11.1 Árbol Binario Extendido con Longitud de Camino con Peso Mínimo

3. Sumamos estos dos valores $4 + 5 = 9$. Luego se eliminan los valores sumados y se agrega el resultado de la suma, reordenando la lista.

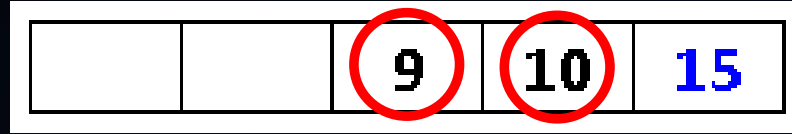
	7	8	9	10
--	---	---	---	----

4. Repetimos el procedimiento anterior con la nueva lista de pesos, seleccionamos los dos pesos mas pequeños o mínimos.
Los sumamos $7 + 8 = 15$. Eliminamos los valores sumados y se agrega el resultado a la lista, reordenándola.

		9	10	15
--	--	---	----	----

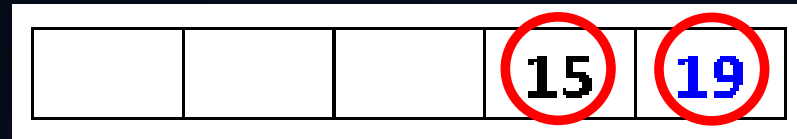
11. Algoritmo de Huffman

11.1 Árbol Binario Extendido con Longitud de Camino con Peso Mínimo

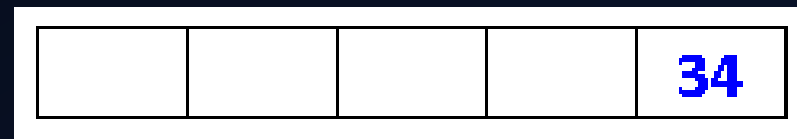


5. Repetimos el procedimiento anterior con la nueva lista de pesos, seleccionamos los dos pesos mas pequeños o mínimos.

Los sumamos $9 + 10 = 19$. Eliminamos los valores sumados y se agrega el resultado a la lista, reordenándola.



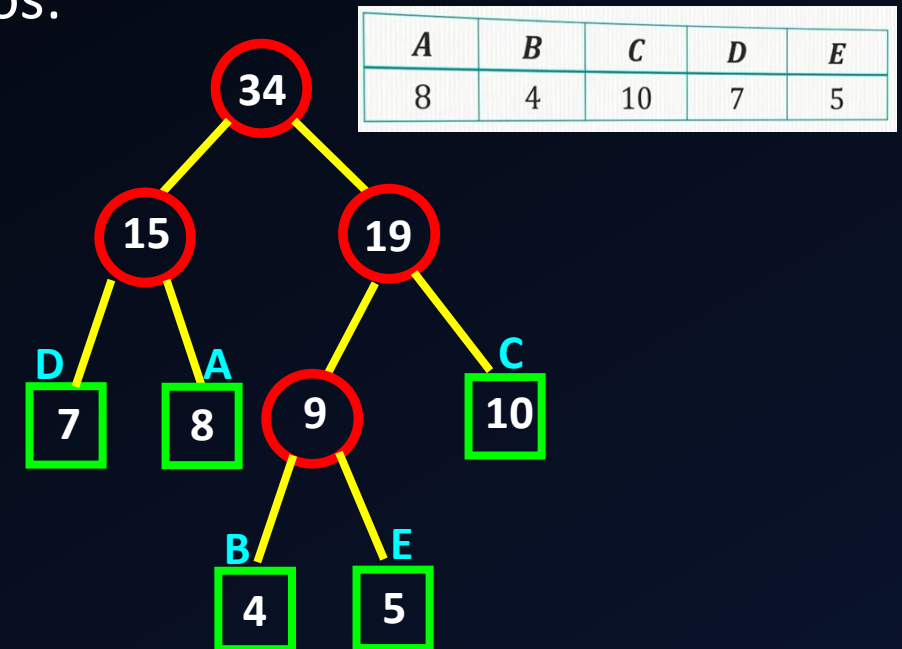
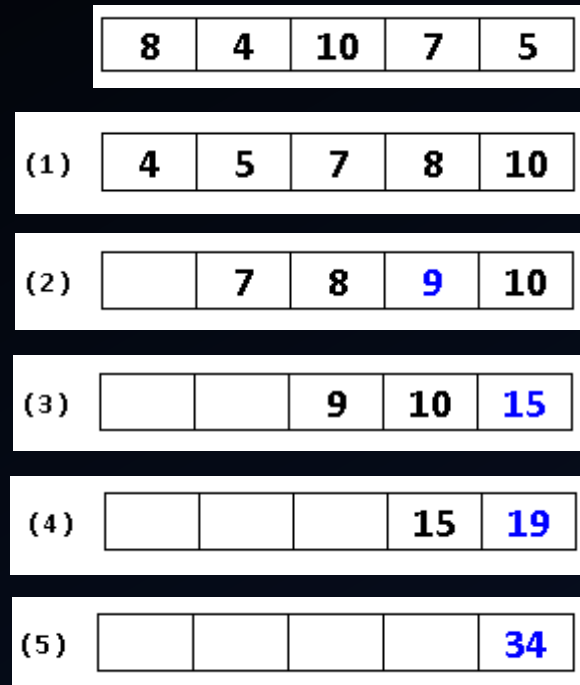
6. En este paso seleccionamos los dos últimos pesos mínimos, Los sumamos $15 + 19 = 34$. Eliminamos los valores sumados y se agrega el ultimo resultado a la lista.



11. Algoritmo de Huffman

11.1 Árbol Binario Extendido con Longitud de Camino con Peso Mínimo

Resumiendo los pasos realizados tenemos:



Después que hemos terminado de realizar el procedimiento anterior, el cual concluye cuando solo nos queda un peso, procedemos a construir el árbol binario extendido con una longitud de camino con peso mínimo.

11. Algoritmo de Huffman

11.1 Árbol Binario Extendido con Longitud de Camino con Peso Mínimo

Practica:

Suponga que A, B, C, D, E, F, G y H son ocho datos y suponga que tienen asignado los siguientes pesos:

A	B	C	D	E	F	G	H
22	5	11	19	2	11	25	5

1. Ordenamos la lista de menor a mayor.

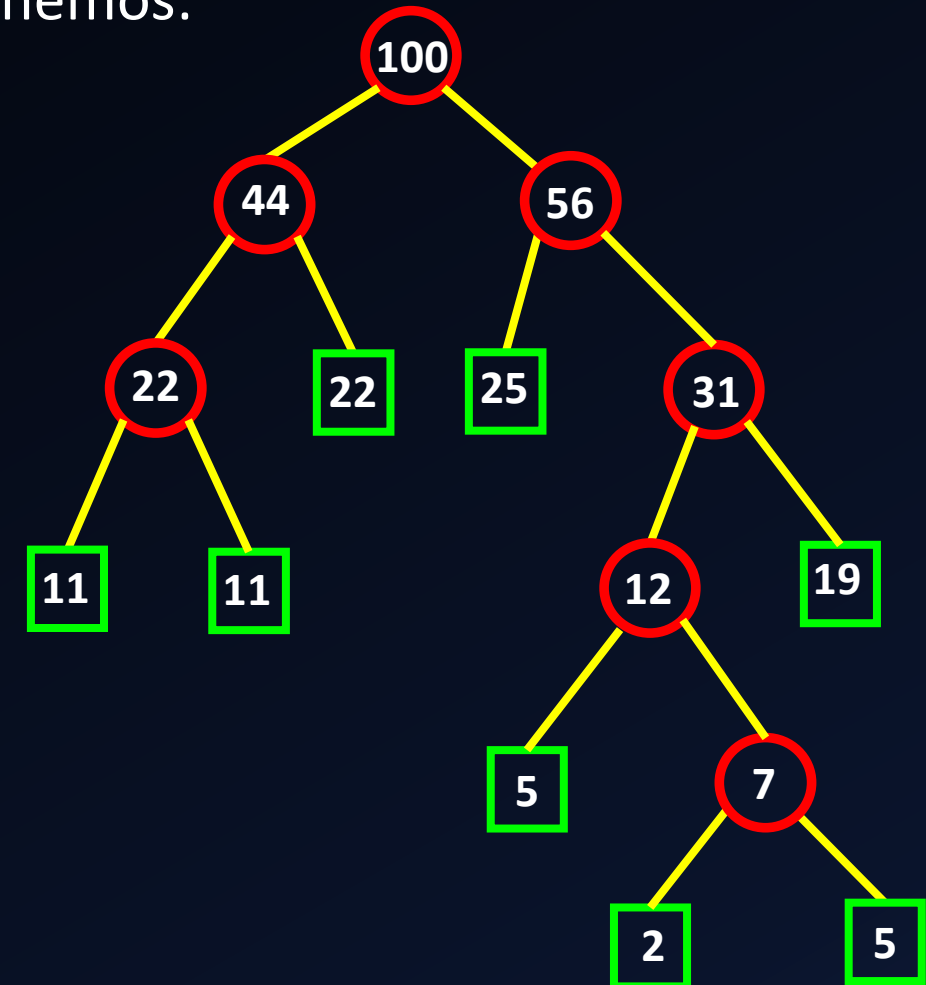
E	B	H	C	F	D	A	G
2	5	5	11	11	19	22	25

11. Algoritmo de Huffman

11.1 Árbol Binario Extendido con Longitud de Camino con Peso Mínimo

Resumiendo los pasos realizados tenemos:

	E	B	H	C	F	D	A	G
(1)	2	5	5	11	11	19	22	25
(2)		5	7	11	11	19	22	25
(3)			11	11	12	19	22	25
(4)				12	19	22	22	25
(5)					22	22	25	31
(6)						25	31	44
(7)							44	56
(8)								100



Después que hemos terminado de realizar el procedimiento anterior, procedemos a construir el árbol binario extendido con una longitud de camino con peso mínimo.

11. Algoritmo de Huffman

11.2 Código Binario de Huffman

Ya sabemos que inicialmente el algoritmo de Huffman consiste en la creación de un Árbol Binario Extendido con Longitud de Camino con Peso Mínimo, donde cada carácter del archivo, estará en los nodos hojas, terminales o externos de este árbol.

La segunda parte del algoritmo de Huffman consiste en recodificar cada uno de estos caracteres, realizando un recorrido desde la raíz hasta el nodo terminal del carácter, podemos obtener el código de Huffman asociado al carácter, pero con una cantidad de bits menores a 8.

Pasos para el Código Binario de Huffman

- 1) Realizar una lista de caracteres, por su frecuencia de aparición, o sea, contar cuantas veces aparece cada carácter en el archivo a comprimir y crear una lista de los caracteres con su frecuencia.

11. Algoritmo de Huffman

11.2 Código Binario de Huffman

Pasos para el Código Binario de Huffman

- 2) Ordenar la lista de menor a mayor en función de la frecuencia y convertir cada elemento de la lista en un árbol de un solo nodo y sin hijos, etiquetando cada nodo con el carácter y la frecuencia de aparición, donde dicha frecuencia sería el peso del nodo.
Si tenemos dos o mas caracteres con la misma frecuencia de aparición, se ordenaría alfabéticamente.
- 3) Fusionar todos estos nodos en un único árbol binario extendido con longitud de camino ponderado o con peso mínimo, de la misma forma como fue explicado anteriormente.
- 4) Codificar cada rama del nuevo árbol binario extendido con un **0** o con un **1**. Si la rama esta a la izquierda del nodo, colocaremos un cero (**0**) y si la rama esta a la derecha colocaremos un uno (**1**).

11. Algoritmo de Huffman

11.2 Código Binario de Huffman

Pasos para el Código Binario de Huffman

- 5) Codificar cada carácter del árbol binario extendido con su nueva codificación de unos y ceros. Para conocer el nuevo el código, a cada carácter en el árbol debemos de realizar un recorrido desde la raíz hasta el carácter, anotando la secuencia de ceros (0) y unos (1). Realizar una lista de cada carácter con la codificación de Huffman.
- 6) Recodificar cada carácter del texto con el nuevo código.
- 7) Calcular el porcentaje de compresión.

11. Algoritmo de Huffman

Práctica

Dada la siguiente frase “**ATA LA VACA A LA ESTACA**”, y luego de pasarla por el algoritmo de Huffman. Encontrar:

- a) La lista de caracteres y cuantas veces se repite en la frase.
- b) La tabla de valores para construir el árbol binario extendido.
- c) Construir el árbol binario extendido con peso mínimo.
- d) La nueva codificación binaria para cada uno de los caracteres.
- e) En cuantos byte se almacena la frase, antes y después de la compresión.
- f) Calcular el porcentaje % de compresión.

11. Algoritmo de Huffman

Práctica

Frase “ATA LA VACA A LA ESTACA”

- 1) Obtener una lista de caracteres con su frecuencia de aparición.

A	T		L	V	C	E	S
9	2	5	2	1	2	1	1

- 2) Ordenar la lista de caracteres de menor a mayor y construir el árbol binario extendido con longitud de camino ponderado mínimo.

1)

E	S	V	C	L	T		A
1	1	1	2	2	2	5	9

2)

	V	C	L	T	E+S		A
	1	2	2	2	2	5	9

3)

		L	T	E+S	V+C		A
		2	2	2	3	5	9

11. Algoritmo de Huffman

Práctica

- 2) Ordenar la lista de caracteres de menor a mayor y construir el árbol binario extendido con longitud de camino ponderado mínimo.

4)

			E + S	V + C	L + T		A
			2	3	4	5	9

5)

				L + T		(E + S) (V + C)	A
				4	5	5	9

6)

					(E + S) (V + C)	A	(L + T)
					5	9	9

7)

						L + T	(E + S) (V + C)
						9	14

8)

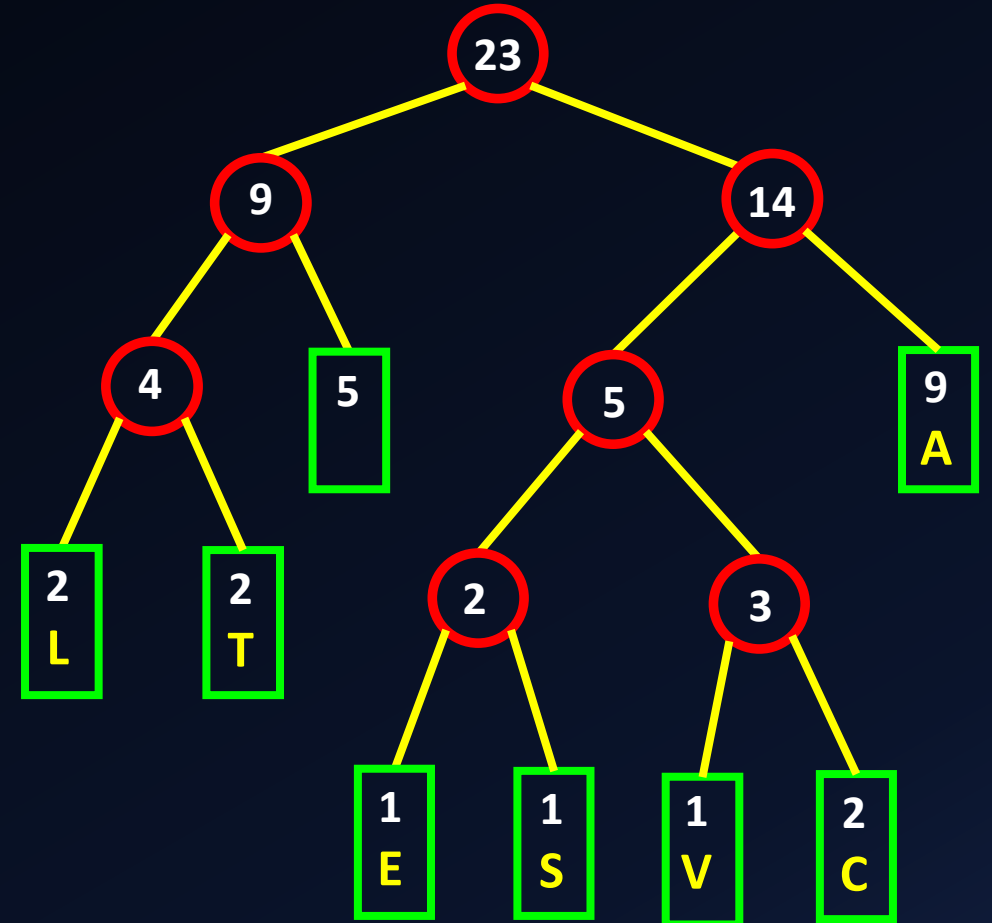
							23
--	--	--	--	--	--	--	----

11. Algoritmo de Huffman

Práctica

a) El árbol binario extendido de la frecuencia de caracteres.

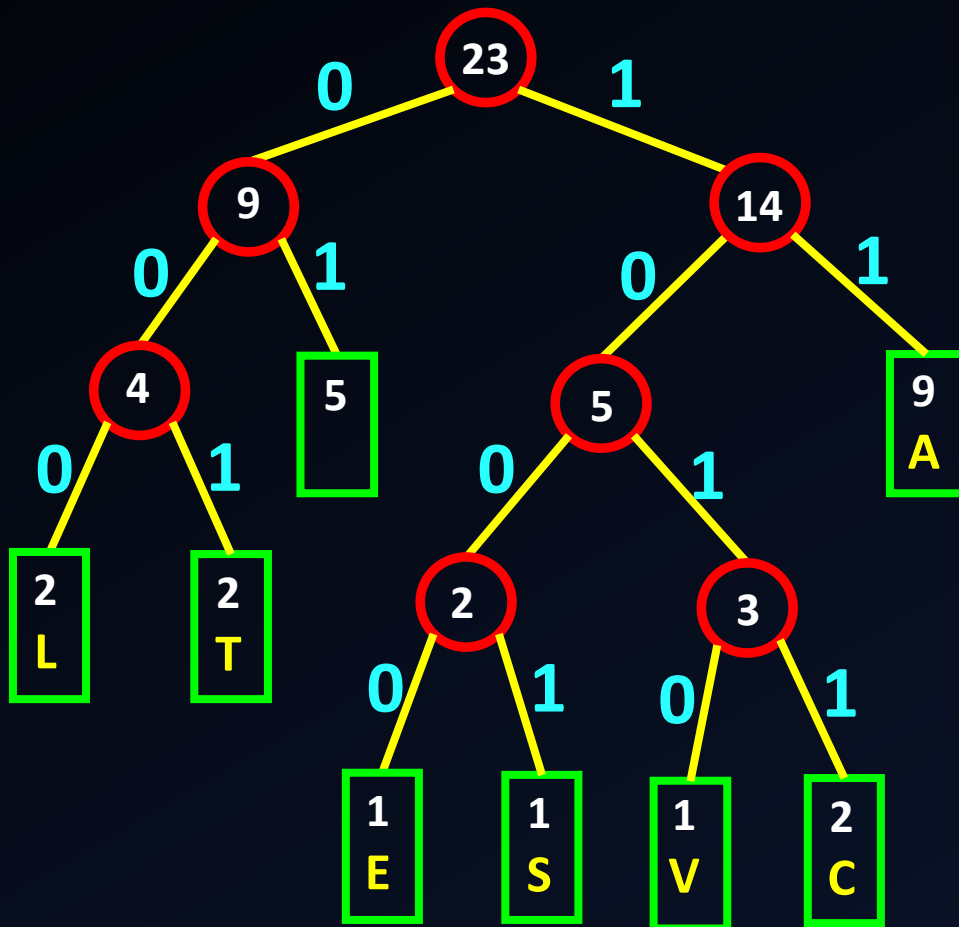
1)	E	S	V	C	L	T		A
	1	1	1	2	2	2	5	9
2)		V	C	L	T	E + S		A
		1	2	2	2	2	5	9
3)			L	T	E + S	V + C		A
			2	2	2	3	5	9
4)				E + S	V + C	L + T		A
				2	3	4	5	9
5)					L + T		(E + S) (v + c)	A
					4	5	5	9
6)						(E + S) (v + c)	A	(L + T)
						5	9	9
7)							(L + T)	(E + S) (v + c)
							9	14
8)								23



11. Algoritmo de Huffman

Práctica

b) La nueva codificación binaria para cada uno de los caracteres.



Carácter	Nuevo Código Binario
L	0 0 0
T	0 0 1
" "	0 1
E	1 0 0 0
S	1 0 0 1
V	1 0 1 0
C	1 0 1 1
A	1 1

11. Algoritmo de Huffman

Práctica

c) En cuantos byte se almacena la frase, antes y después de la compresión.

Carácter	Nuevo Código Binario	Repeticiones	Nueva Cantidad de BITS
L	0 0 0	2	6
T	0 0 1	2	6
" "	0 1	5	10
E	1 0 0 0	1	4
S	1 0 0 1	1	4
V	1 0 1 0	1	4
C	1 0 1 1	2	8
A	1 1	9	18

Total de Bits



60 bits

ATA LA VACA A LA ESTACA

A	T		L	V	C	E	S
9	2	5	2	1	2	1	1

Cantidad de Bytes antes de la compresión

23 Bytes

Cantidad de Bytes después de la compresión

$$60 / 8 = 7.5$$

8 Bytes

11. Algoritmo de Huffman

Práctica

d) Calcular el porcentaje % de compresión.

ATA LA VACA A LA ESTACA

Cantidad de Bytes antes de la compresión

23 Bytes

Cantidad de Bytes después de la compresión

8 Bytes

23 – 8 = 15 Bytes menos

Regla de Tres

23 → 100%

15 → X

$$\frac{15 \times 100}{23} = 65.22 \%$$

11. Algoritmo de Huffman

Práctica

Dada la siguiente frase “**TRES TRISTES TIGRES COMEN TRIGO EN EL TRIGAL**”, y luego de pasarla por el algoritmo de Huffman. Encontrar:

- a) La lista de caracteres y cuantas veces se repite en la frase.
- b) La tabla de valores para construir el árbol binario extendido.
- c) Construir el árbol binario extendido con peso mínimo.
- d) La nueva codificación binaria para cada uno de los caracteres.
- e) En cuantos byte se almacena la frase, antes y después de la compresión.
- f) Calcular el porcentaje % de compresión.

Gracias.....